Introduction to machine learning

1

Requirements:

- Practice 70% (notebook submission)
- 40% + on the lecture test
- Project and presentation

Grade

- Homework / 10
- Test / 40
- Project / 100

$\Sigma = 140$ 85%, 70%, ...

Subjects

- k Nearest neighbor
- Decision tree
- Random forest / ensembles
- Lin / log regression
- Support vector machine
- Dense neural network I / II
- Convolutional NN
- Principal component analysis
- Maximum likelihood *
- K-means
- Genetic algorithm

Learning

- process of gaining knowledge
- learn from experience E, with respect to task T and performance measure P

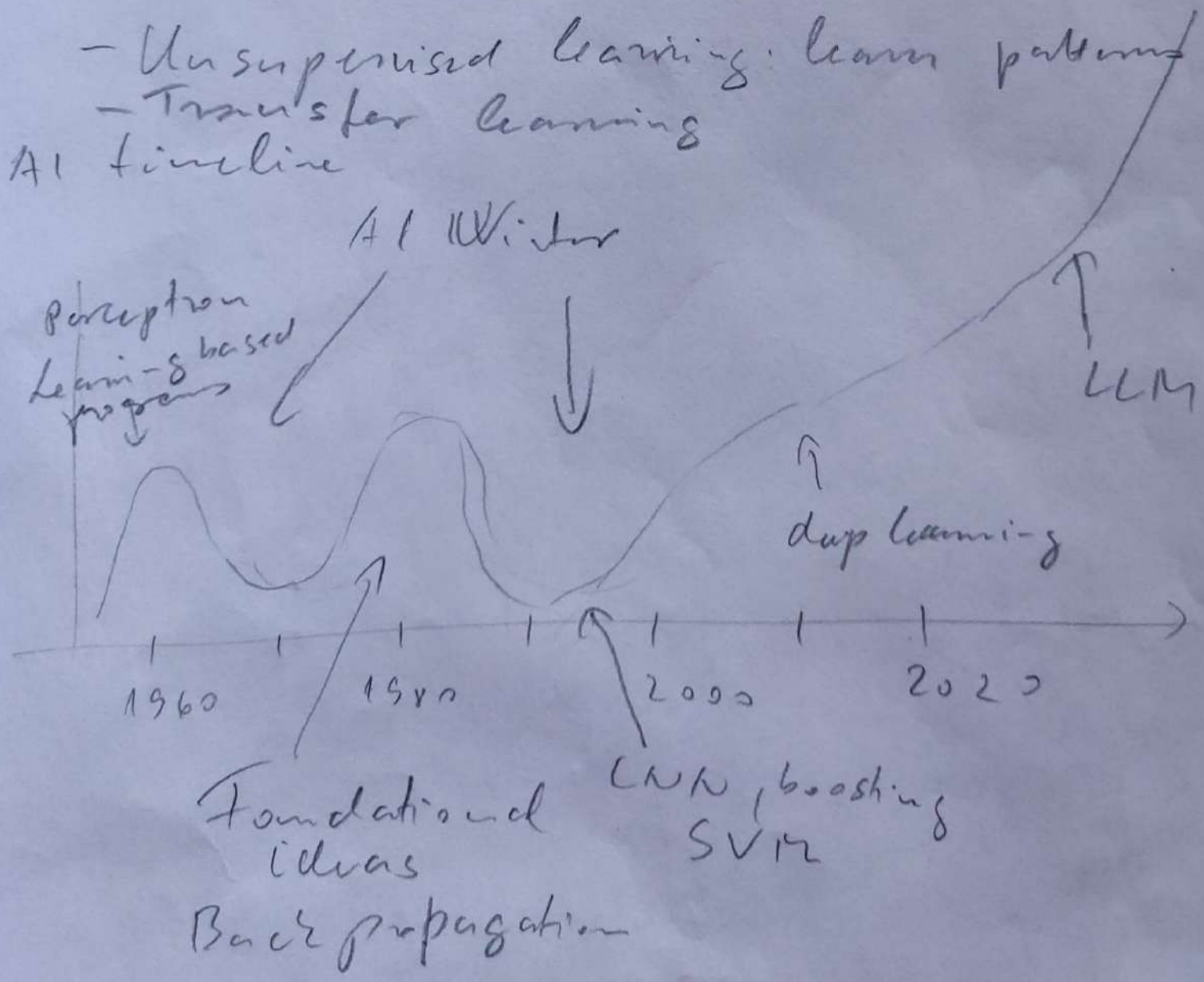
- learn a lot ≠ being intelligent

ML ≠ AI

Learning types

- Supervised learning: (labelled training data)
- Reinforcement learning: maximise reward based on performance
- Unsupervised learning: learn patterns
- Transfer learning

AI timeline



k-Nearest Neighbor

Supervised learning: training set
↳ inputs, labels $(\underline{x}^{(i)}, y^{(i)})$

Goal: predict labels for data not in training set

Input vector: $\in \mathbb{R}^d$
↳ called representation

Hope: distance makes sense !!

Representations

- numerical data: as is
- image: pixels as RGB
- words: unique codes, e.g.: cat: 70
  objects dog: 87
  ...

⊗ One hot encoding: length of the vector = length of the token set, All values are zero except the one representing the objects

0	0	0	1	0	0

⊕ a vector incorporating its meaning

Target or label types

- Regression $t \in \mathbb{R}$
- Classification $t \in \{1, \dots C\}$ discrete

Training set : $(\{\underline{x}^{(1)}, t^{(1)}\}, \{x^{(2)}, t^{(2)}\} \dots \{x^{(n)}, t^{(n)}\})$

Distance

$d(\underline{x}^{(i)}, \underline{x}^{(j)}) = ?$ $\quad \underline{x} \in \mathbb{R}^d$

- Euclidean :

$$\| \underline{x}^{(i)}, \underline{x}^{(j)} \| = \sqrt{\sum_{\alpha=1}^{d} (x_\alpha^{(i)} - x_\alpha^{(j)})^2}$$

- Hamming distance

$$H(\underline{x}^{(i)}, \underline{x}^{(j)}) = d - \sum_{\alpha=1}^{d} \delta(x_\alpha^{(i)}, x_\alpha^{(j)})$$

Number of positions the two vector differs

- Cosine distance

$$1 - \text{sim}(\underline{x}^{(i)}, \underline{x}^{(j)}) = 1 - \frac{\underline{x}^{(i)} \cdot \underline{x}^{(j)}}{\| \underline{x}^{(i)} \| \, \| \underline{x}^{(j)} \|}$$

Voronoi - cells / diagrams

$$R_i = \{ x \in X \mid d(x, P_i) \leq d(x, P_j) \ \forall j \neq i \}$$

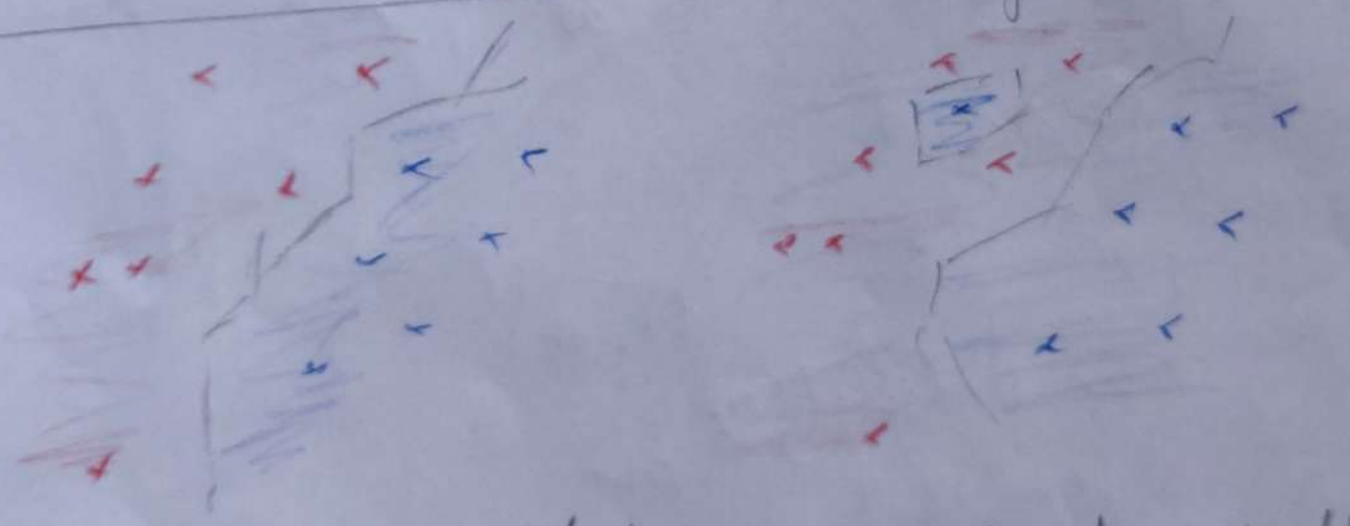
domain of point P_i



In our case

$$t^* = \arg\min_i d(x^*, x^{(i)})$$

Decision boundary: problems



k - nearest neighbor: majority of the k nearest neighbor

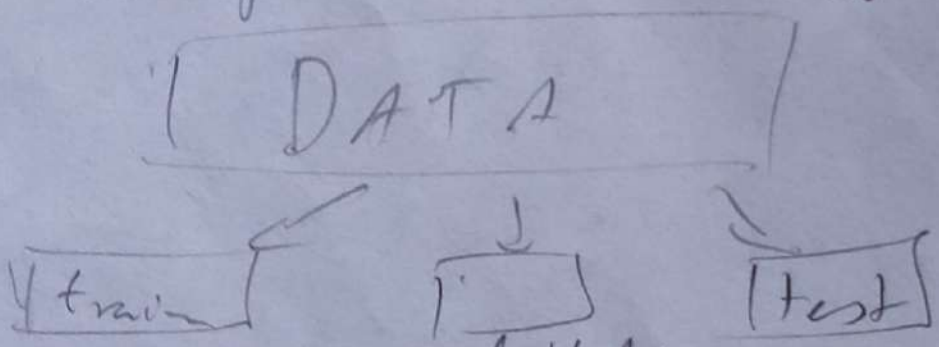
Small k
- fine pattern
- overfitting

Large k
- stable pattern
- underfitting

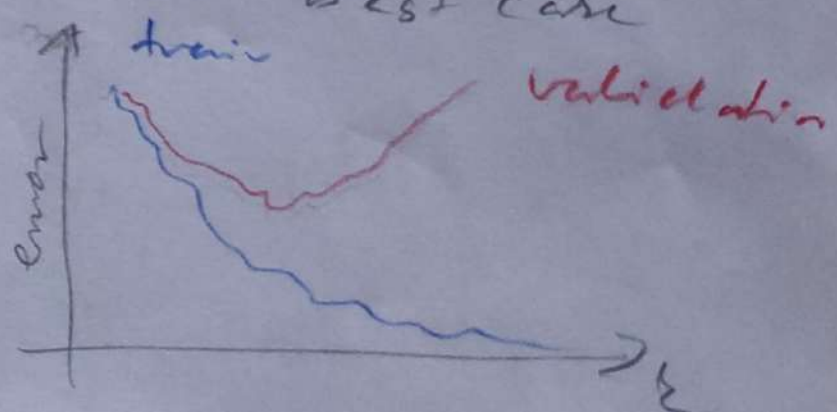
popular choice: $k < \sqrt{n}$ n number of training objects

Hyperparameter
- a parameter the algorithm cannot fit
- controls the behaviour of the algorithm

? How to optimize a hyperparameter?

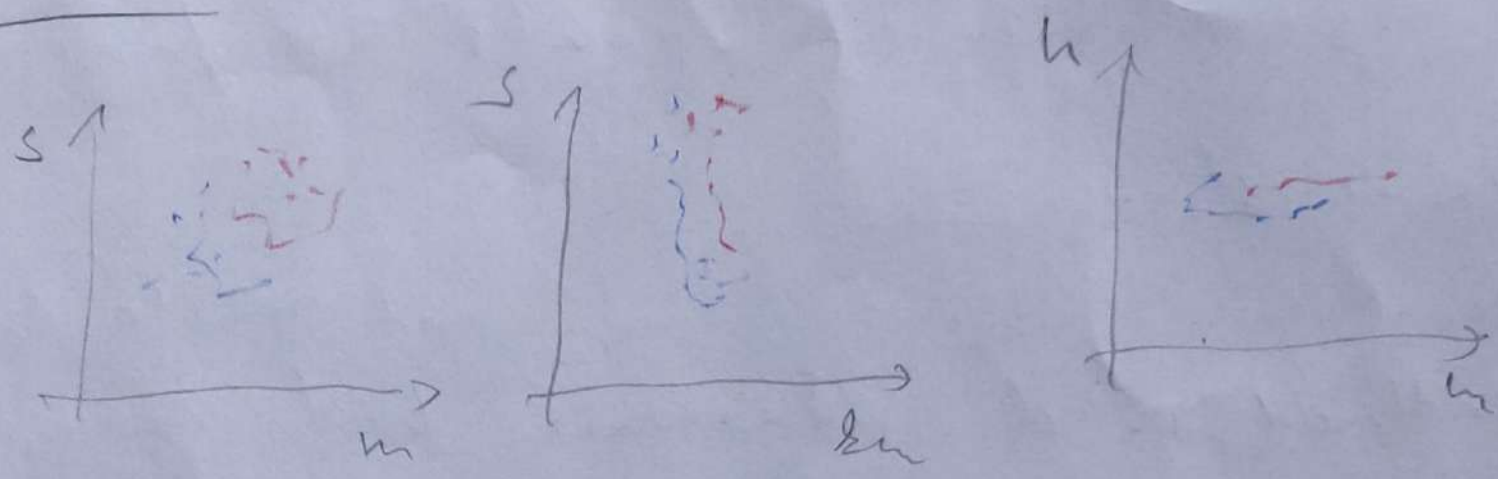


train	$k=1$	err 2,7	err
	$k=2$	1,3	
	$k=3$	0,5	→ 1,3 only for the best case
	$k=4$	1,1	
	⋮		
	$k=50$	9,3	



- 1N14 -

Units



Data normalization

$$\tilde{x}_\alpha^{(i)} = \frac{x_\alpha^{(i)} - \mu_\alpha}{\sigma_\alpha}$$ → zero mean, unit variance

Important

- different units
- parameter importance

Dimensionality

dim = d

E.g. data in unit cube $V_{tot} = 1$

Cover it with N cubes of size $r < 1$

$V_a = r^d$ $\quad r = N^{-1/d}$

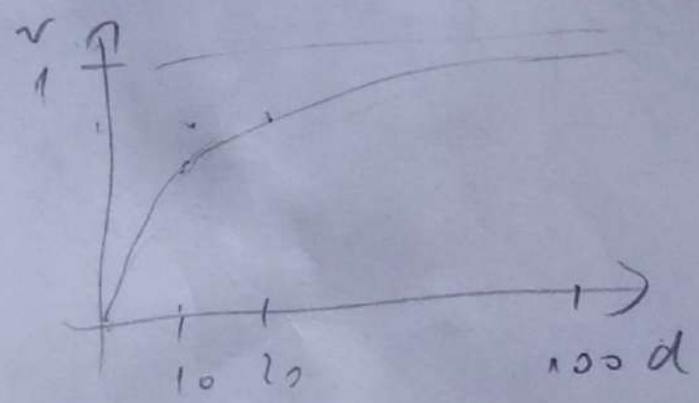
N = 100

d = 2 r = 0,1

d = 10 r = 0,63

d = 20 r = 0,75

d = 100 r = 0,95



Consequence

In high dimension points are far away

Intrinsic dimension

- sometimes objects are not randomly scattered in d dimension but occupy some subspace with much lower dimension.

e.g. images → neighboring pixels have similar colors, and color range is not random

Summary of kNN

- training time : ~~0~~
- test time
  - distance $d \cdot N$
  - sort $N \ln(N)$

Best for low dimension and small training set